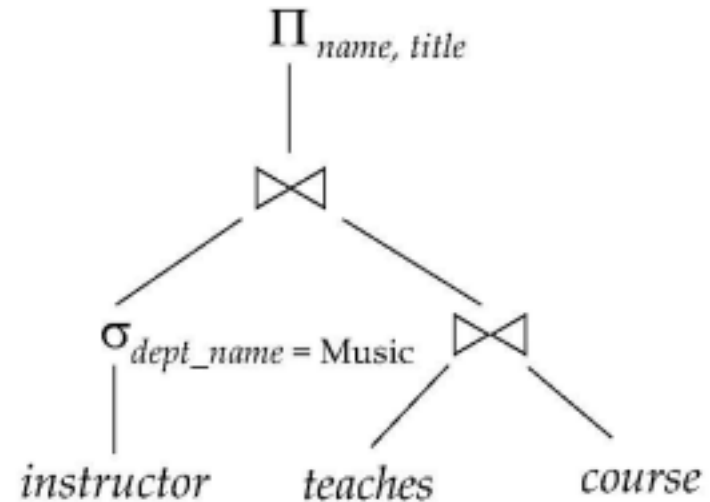
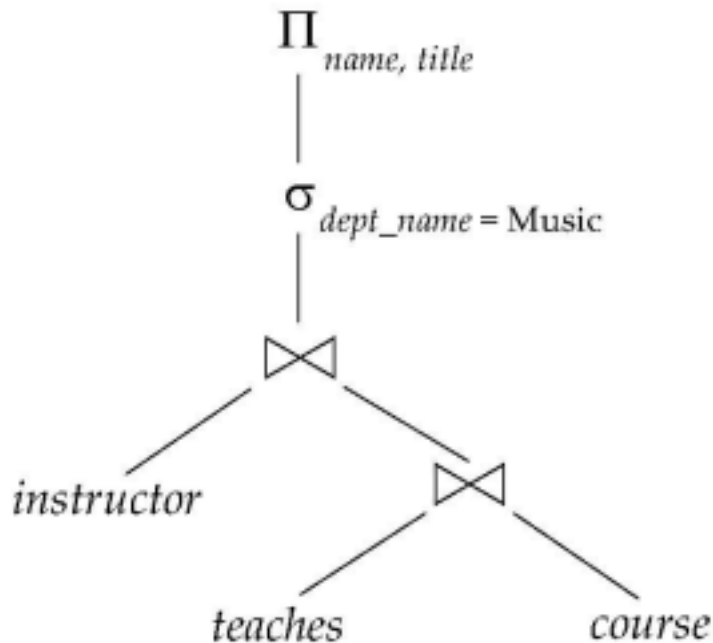


Chapter 13: Query Optimization

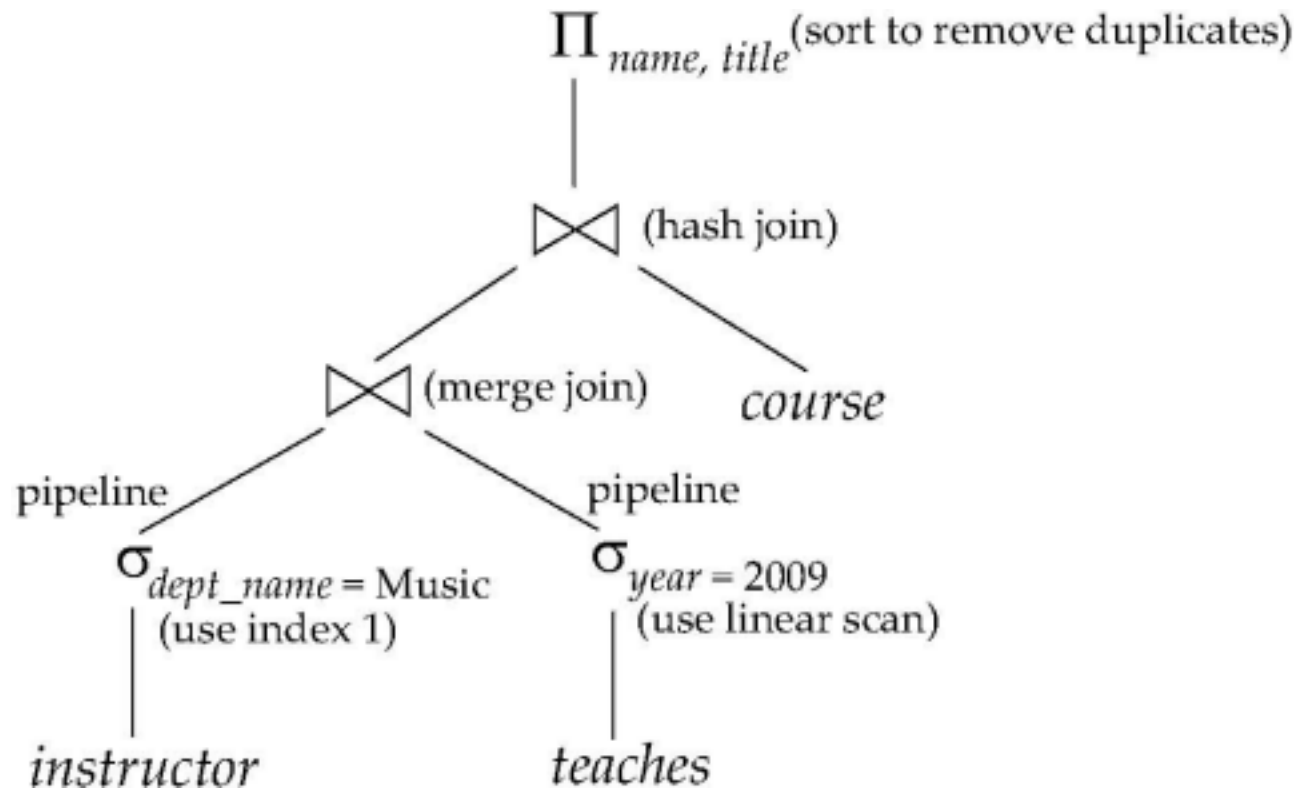
Introduction

- Alternative ways of evaluating a given query
 - Equivalent expressions
 - Different algorithms for each operation



Introduction (Cont.)

- An **evaluation plan** defines exactly what algorithm is used for each operation, and how the execution of the operations is coordinated.



- Find out how to view query execution plans on your favorite database

Introduction (Cont.)

- Cost difference between evaluation plans for a query can be enormous
 - E.g. seconds vs. days in some cases
- Steps in **cost-based query optimization**
 1. Generate logically equivalent expressions using **equivalence rules**
 2. Annotate resultant expressions to get alternative query plans
 3. Choose the cheapest plan based on **estimated cost**
- Estimation of plan cost based on:
 - Statistical information about relations. Examples:
 - ▶ number of tuples, number of distinct values for an attribute
 - Statistics estimation for intermediate results
 - ▶ to compute cost of complex expressions
 - Cost formulae for algorithms, computed using statistics

Generating Equivalent Expressions

Transformation of Relational Expressions

- Two relational algebra expressions are said to be **equivalent** if the two expressions generate the same set of tuples on every *legal* database instance
 - Note: order of tuples is irrelevant
 - we don't care if they generate different results on databases that violate integrity constraints
- In SQL, inputs and outputs are multisets of tuples
 - Two expressions in the multiset version of the relational algebra are said to be equivalent if the two expressions generate the same multiset of tuples on every legal database instance.
- An **equivalence rule** says that expressions of two forms are equivalent
 - Can replace expression of first form by second, or vice versa

Equivalence Rules

1. Conjunctive selection operations can be deconstructed into a sequence of individual selections.

$$\sigma_{\theta_1 \wedge \theta_2}(E) = \sigma_{\theta_1}(\sigma_{\theta_2}(E))$$

2. Selection operations are commutative.

$$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) = \sigma_{\theta_2}(\sigma_{\theta_1}(E))$$

3. Only the last in a sequence of projection operations is needed, the others can be omitted.

$$\Pi_{L_1}(\Pi_{L_2}(\dots(\Pi_{L_n}(E))\dots)) = \Pi_{L_1}(E)$$

4. Selections can be combined with Cartesian products and theta joins.

$$\alpha. \sigma_{\theta}(E_1 \times E_2) = E_1 \theta E_2$$

$$\beta. \sigma_{\theta_1}(E_1 \theta_2 E_2) = E_1 \theta_{1 \wedge \theta_2} E_2$$

Equivalence Rules (Cont.)

5. Theta-join operations (and natural joins) are commutative.

$$E_1 \theta E_2 = E_2 \theta E_1$$

6. (a) Natural join operations are associative:

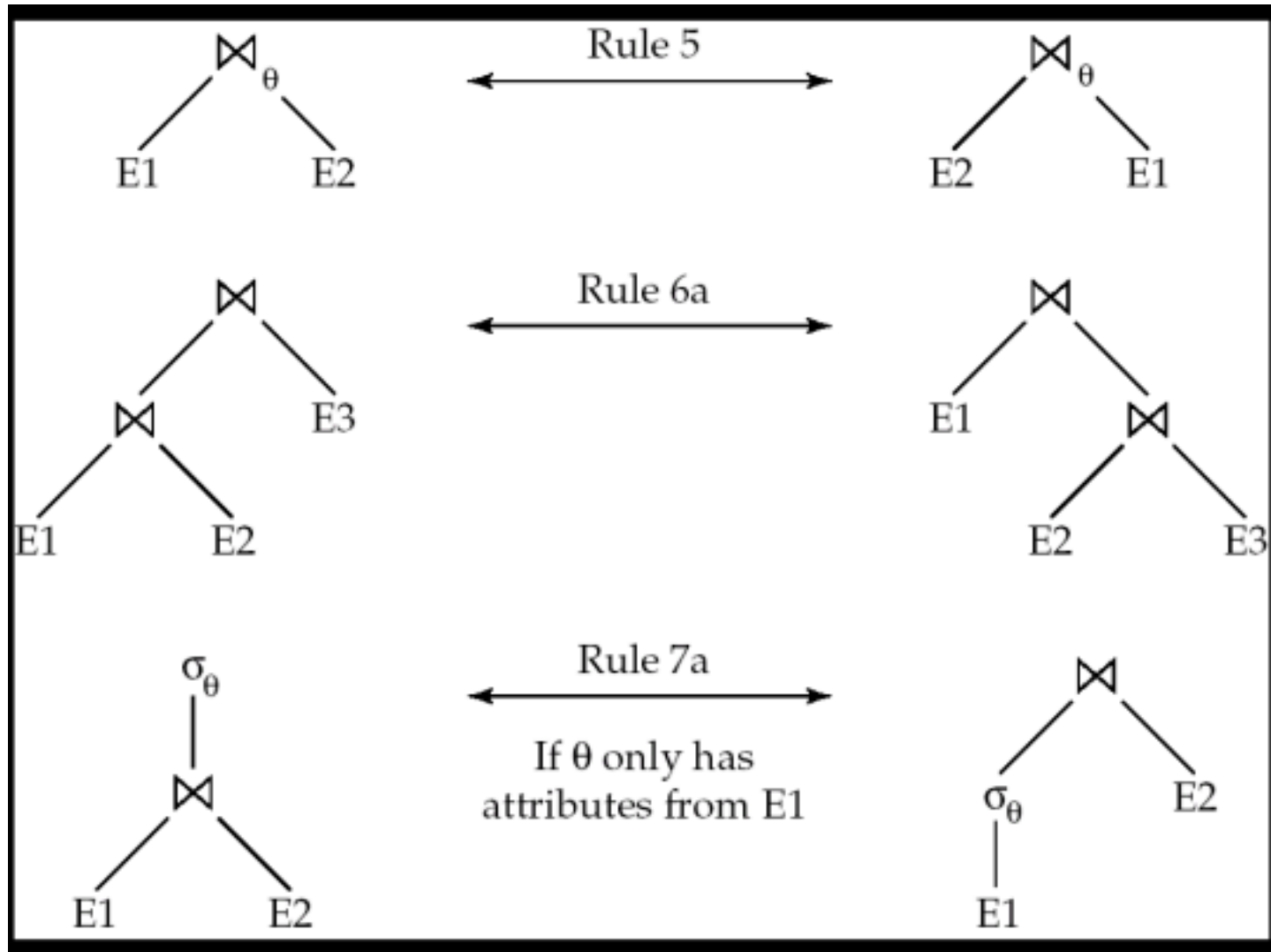
$$(E_1 E_2) E_3 = E_1 (E_2 E_3)$$

- (b) Theta joins are associative in the following manner:

$$(E_1 \theta_1 E_2) \theta_{2 \wedge \theta_3} E_3 = E_1 \theta_{1 \wedge \theta_3} (E_2 \theta_2 E_3) \quad \text{where}$$

θ_2 involves attributes from only E_2 and E_3 .

Pictorial Depiction of Equivalence Rules



Equivalence Rules (Cont.)

7. The selection operation distributes over the theta join operation under the following two conditions:

(a) When all the attributes in θ_0 involve only the attributes of one of the expressions (E_1) being joined.

$$\sigma_{\theta_0}(E_1 \theta E_2) = (\sigma_{\theta_0}(E_1)) \theta E_2$$

(b) When θ_1 involves only the attributes of E_1 and θ_2 involves only the attributes of E_2 .

$$\sigma_{\theta_1 \wedge \theta_2}(E_1 \theta E_2) = (\sigma_{\theta_1}(E_1)) \theta (\sigma_{\theta_2}(E_2))$$

Equivalence Rules (Cont.)

8. The projection operation distributes over the theta join operation as follows: (a) if θ involves only attributes from $L_1 \cup L_2$:

$$\pi_{L_1 \cup L_2}((E_1 \theta E_2)) = (\pi_{L_1}(E_1) \theta \pi_{L_2}(E_2))$$

(b) Consider a join $E_1 \theta E_2$.

- Let L_1 and L_2 be sets of attributes from E_1 and E_2 , respectively. – Let L_3 be attributes of E_1 that are involved in join condition θ , but are not in $L_1 \cup L_2$, and
- let L_4 be attributes of E_2 that are involved in join condition θ , but are not

in $L_1 \cup L_2$.

$$((\sigma_{\theta}(\sigma_{\theta}(E_1 \cup E_2)) \cap \sigma_{\theta}(E_1)) \cap \sigma_{\theta}(E_2)) = \sigma_{\theta}(\sigma_{\theta}(E_1) \cap \sigma_{\theta}(E_2))$$

Equivalence Rules (Cont.)

9. The set operations union and intersection are commutative

$$E_1 \cup E_2 = E_2 \cup E_1$$

$$E_1 \cap E_2 = E_2 \cap E_1$$

■ (set difference is not commutative).

10. Set union and intersection are associative.

$$(E_1 \cup E_2) \cup E_3 = E_1 \cup (E_2 \cup E_3)$$

$$(E_1 \cap E_2) \cap E_3 = E_1 \cap (E_2 \cap E_3)$$

11. The selection operation distributes over $\cup$, $\cap$ and $-$.

$$\sigma_{\theta}(E_1 - E_2) = \sigma_{\theta}(E_1) - \sigma_{\theta}(E_2)$$

and similarly for $\cup$ and $\cap$ in place of $-$ Also: $\sigma_{\theta}(E_1 - E_2) = \sigma_{\theta}(E_1) - E_2$

and similarly for $\cap$ in place of $-$, but not for $\cup$ 12. The

projection operation distributes over union $\Pi_L(E_1 \cup E_2) = (\Pi_L(E_1)) \cup (\Pi_L(E_2))$

Transformation Example: Pushing Selections

- Query: Find the names of all instructors in the Music department, along with the titles of the courses that they teach

– $\Pi_{name, title}(\sigma_{dept_name = \text{"Music"}}(instructor \text{ teaches } \Pi_{course_id, title}(course)))$

- Transformation using rule 7a.

– $\Pi_{name, title}((\sigma_{dept_name = \text{"Music"}}(instructor)) \text{ teaches } \Pi_{course_id, title}(course))$

- Performing the selection as early as possible reduces the size of the relation to be joined.

Example with Multiple Transformations

- **Query:** Find the names of all instructors in the Music department who have taught a course in 2009, along with the titles of the courses that they taught

– $\Pi_{name, title}(\sigma_{dept_name = \text{“Music”} \wedge gear = 2009}$

$(instructor\ teaches\ \Pi_{course_id, title}(course)))$

- Transformation using join associatively (Rule 6a):

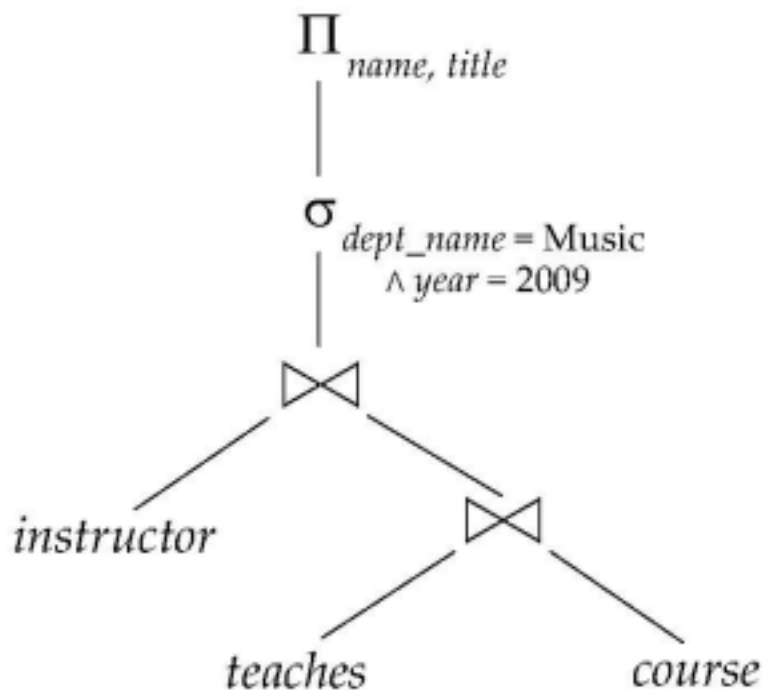
– $\Pi_{name, title}(\sigma_{dept_name = \text{“Music”} \wedge gear = 2009}$

$((instructor\ teaches)\ \Pi_{course_id, title}(course)))$

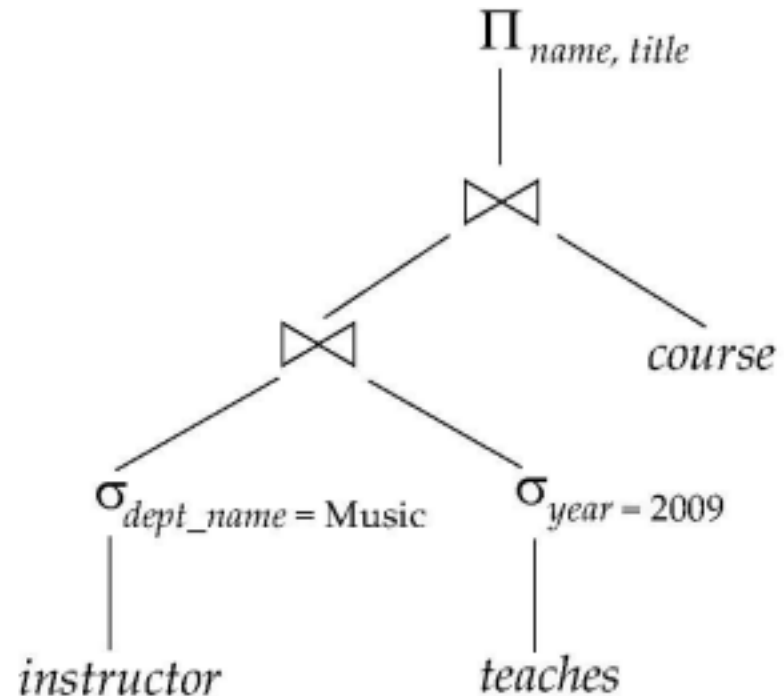
- Second form provides an opportunity to apply the “perform selections early” rule, resulting in the subexpression

$\sigma_{dept_name = \text{“Music”}}(instructor)\ \sigma_{year = 2009}(teaches)$

Multiple Transformations (Cont.)



(a) Initial expression tree



(b) Tree after multiple transformations

Transformation Example: Pushing Projections

- Consider: $\Pi_{name, title}(\sigma_{dept_name = \text{"Music"}}(instructor) \text{ teaches})$
 $\Pi_{course_id, title}(course))$

- When we compute

$$(\sigma_{dept_name = \text{"Music"}}(instructor\ teaches))$$

we obtain a relation whose schema is:

$(ID, name, dept_name, salary, course_id, sec_id, semester, year)$

- Push projections using equivalence rules 8a and 8b; eliminate unneeded attributes from intermediate results to get:

$$\Pi_{name, title}(\Pi_{name, course_id}(\sigma_{dept_name = \text{"Music"}}(instructor\ teaches)) \Pi_{course_id, title}(course))))$$

- Performing the projection as early as possible reduces the size of the relation to be joined.

Join Ordering Example

- For all relations r_1, r_2 , and r_3 ,

$$(r_1 r_2) r_3 = r_1 (r_2 r_3)$$

(Join Associativity)

- If $r_2 r_3$ is quite large and $r_1 r_2$ is small, we choose

$$(r_1 r_2) r_3$$

so that we compute and store a smaller temporary relation.

Join Ordering Example (Cont.)

- Consider the expression

$$\Pi_{name, title}(\sigma_{dept_name = \text{"Music"}}(instructor) \text{ teaches } \Pi_{course_id, title}(course)))$$

- Could compute $teaches \Pi_{course_id, title}(course)$ first, and join result with

$$\sigma_{dept_name = \text{"Music"}}(instructor)$$

but the result of the first join is likely to be a large relation.

- Only a small fraction of the university's instructors are likely to be from the Music department
 - it is better to compute

$$\sigma_{dept_name = \text{"Music"}}(instructor) \text{ teaches}$$

first.

Enumeration of Equivalent Expressions •

Query optimizers use equivalence rules to **systematically** generate expressions equivalent to the given expression •

Can generate all equivalent expressions as follows: – Repeat

- apply all applicable equivalence rules on every subexpression of every

equivalent expression found so far

- add newly generated expressions to the set of equivalent expressions

Until no new equivalent expressions are generated above

- The above approach is very expensive in space and time
 - Two approaches
 - Optimized plan generation based on transformation rules
 - Special case approach for queries with only selections, projections and joins

Implementing Transformation Based Optimization

- Space requirements reduced by sharing common sub-expressions:
 - when E1 is generated from E2 by an equivalence rule, usually only the top level of the two are different, subtrees below are the same and can be shared using pointers • E.g. when applying join commutativity

E1 E2

- Same sub-expression may get generated multiple times
 - Detect duplicate sub-expressions and share one copy

- Time requirements are reduced by not generating all expressions
 - Dynamic programming
 - We will study only the special case of dynamic programming for join order optimization

Cost Estimation

- Cost of each operator computer as described in Chapter 12
 - Need statistics of input relations
 - E.g. number of tuples, sizes of tuples
- Inputs can be results of sub-expressions
 - Need to estimate statistics of expression results
 - To do so, we require additional statistics
 - E.g. number of distinct values for an attribute
- More on cost estimation later

Choice of Evaluation Plans

- Must consider the interaction of evaluation techniques when choosing evaluation plans
 - choosing the cheapest algorithm for each operation independently may not yield best overall algorithm. E.g.
 - merge-join may be costlier than hash-join, but may provide a sorted output which reduces the cost for an outer level aggregation.

- nested-loop join may provide opportunity for pipelining
- Practical query optimizers incorporate elements of the following two broad approaches:
 1. Search all the plans and choose the best plan in a cost-based fashion.
 2. Uses heuristics to choose a plan.

Cost-Based Optimization

- Consider finding the best join-order for $r_1 r_2 \dots r_n$. • There are $(2(n-1))!/(n-1)!$ different join orders for above expression. With $n = 7$, the number is 665280, with $n = 10$, the number is greater than 176 billion!
- No need to generate all the join orders. Using dynamic programming, the least-cost join order for any subset of $\{r_1, r_2, \dots, r_n\}$ is computed only once and stored for future use.

Dynamic Programming in Optimization

- To find best join tree for a set of n relations:
 - To find best plan for a set S of n relations, consider all possible plans of the form: $S_1 (S - S_1)$ where S_1 is any non-empty subset of S .
 - Recursively compute costs for joining subsets of S to find the cost of each plan. Choose the cheapest of the $2^n - 2$ alternatives. – Base case for recursion: single relation access plan
 - Apply all selections on R_i using best choice of indices on R_i
 - When plan for any subset is computed, store it and reuse it when it is required again, instead of recomputing it
 - Dynamic programming

Join Order Optimization Algorithm

```

procedure findbestplan( $S$ )
  if ( $bestplan[S].cost \neq \infty$ )
    return  $bestplan[S]$ 
  // else  $bestplan[S]$  has not been computed earlier, compute it now
  if ( $S$  contains only 1 relation)

```

set *bestplan*[*S*].*plan* and *bestplan*[*S*].*cost* based on the best way of accessing *S*

/ Using selections on S and indices on S */*

else for each non-empty subset *S1* of *S* such that *S1* $\neq$ *S*

P1 = *findbestplan*(*S1*)

P2 = *findbestplan*(*S* - *S1*)

A = best algorithm for joining results of *P1* and *P2*

cost = *P1.cost* + *P2.cost* + cost of *A*

if *cost* < *bestplan*[*S*].*cost*

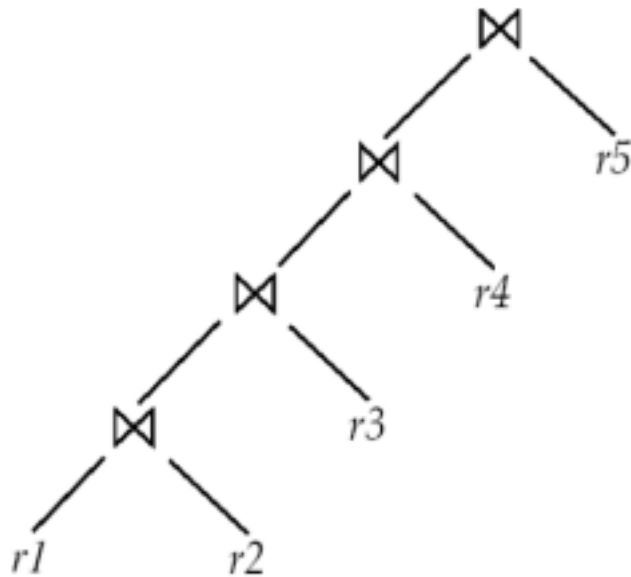
bestplan[*S*].*cost* = *cost*

bestplan[*S*].*plan* = “execute *P1.plan*; execute *P2.plan*; join results of *P1* and *P2* using *A*”

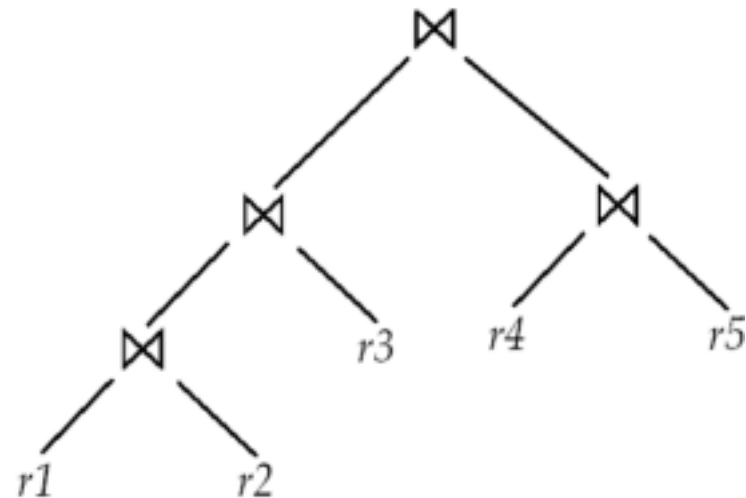
return *bestplan*[*S*]

Left Deep Join Trees

- In **left-deep join trees**, the right-hand-side input for each join is a relation, not the result of an intermediate join.



(a) Left-deep join tree



(b) Non-left-deep join tree

Cost of Optimization

- With dynamic programming time complexity of optimization with bushy trees is $O(3^n)$.
 - With $n = 10$, this number is 59000 instead of 176 billion!
- Space complexity is $O(2^n)$
- To find best left-deep join tree for a set of n relations: –

Consider n alternatives with one relation as right-hand side input and the other relations as left-hand side input.

– Modify optimization algorithm:

- Replace “**for each** non-empty subset S_1 of S such that $S_1 \neq S$ ”
- By: **for each** relation r in S
let $S_1 = S - r$.

- If only left-deep trees are considered, time complexity of finding best join order is $O(n 2^n)$
 - Space complexity remains at $O(2^n)$
- Cost-based optimization is expensive, but worthwhile for queries on large datasets (typical queries have small n , eneral < 10)

Interesting Sort Orders

- Consider the expression $(r_1 r_2) r_3$ (with A as common attribute)
- An **interesting sort order** is a particular sort order of tuples that could be useful for a later operation

- Using merge-join to compute $r_1 r_2$ may be costlier than hash join but generates result sorted on A
- Which in turn may make merge-join with r_3 cheaper, which may reduce cost of join with r_3 and minimizing overall cost
- Sort order may also be useful for order by and for grouping •

Not sufficient to find the best join order for each subset of the set of n given relations

- must find the best join order for each subset, **for each interesting sort order**
- Simple extension of earlier dynamic programming algorithms – Usually, number of interesting orders is quite small and doesn't affect time/space complexity significantly

Cost Based Optimization with Equivalence Rules •

Physical equivalence rules allow logical query plan to be converted to physical query plan specifying what algorithms are used for each operation.

- Efficient optimizer based on equivalent rules depends on
 - A space efficient representation of expressions which avoids

making multiple copies of subexpressions

- Efficient techniques for detecting duplicate derivations of expressions
- A form of dynamic programming based on **memoization**, which stores the best plan for a subexpression the first time it is optimized, and reuses it on repeated optimization calls on same subexpression
- Cost-based pruning techniques that avoid generating all plans •

Pioneered by the Volcano project and implemented in the SQL Server optimizer

Heuristic Optimization

- Cost-based optimization is expensive, even with dynamic programming.
 - Systems may use *heuristics* to reduce the number of choices that must be made in a cost-based fashion. •
- Heuristic optimization transforms the query-tree by using a set of rules that typically (but not in all cases) improve

execution performance:

- Perform selection early (reduces the number of tuples) –
- Perform projection early (reduces the number of attributes) –
- Perform most restrictive selection and join operations (i.e. with smallest result size) before other similar operations.
- Some systems use only heuristics, others combine heuristics with partial cost-based optimization.

Structure of Query Optimizers

- Many optimizers considers only left-deep join orders. – Plus heuristics to push selections and projections down the query tree
 - Reduces optimization complexity and generates plans amenable to pipelined evaluation.
- Heuristic optimization used in some versions of Oracle:
 - Repeatedly pick “best” relation to join next
 - Starting from each of n starting points. Pick best among these
- Intricacies of SQL complicate query optimization –
E.g. nested subqueries

Structure of Query Optimizers (Cont.)

- Some query optimizers integrate heuristic selection and the generation of alternative access plans.
 - Frequently used approach
 - heuristic rewriting of nested block structure and aggregation • followed by cost-based join-order optimization for each block
 - Some optimizers (e.g. SQL Server) apply transformations to entire query and do not depend on block structure
 - **Optimization cost budget** to stop optimization early (if cost of plan is less than cost of optimization)
 - **Plan caching** to reuse previously computed plan if query is resubmitted
 - Even with different constants in query
- Even with the use of heuristics, cost-based query optimization imposes a substantial overhead.
 - But is worth it for expensive queries
 - Optimizers often use simple heuristics for very cheap queries, and perform exhaustive enumeration for more expensive queries

Statistics for Cost Estimation

Statistical Information for Cost

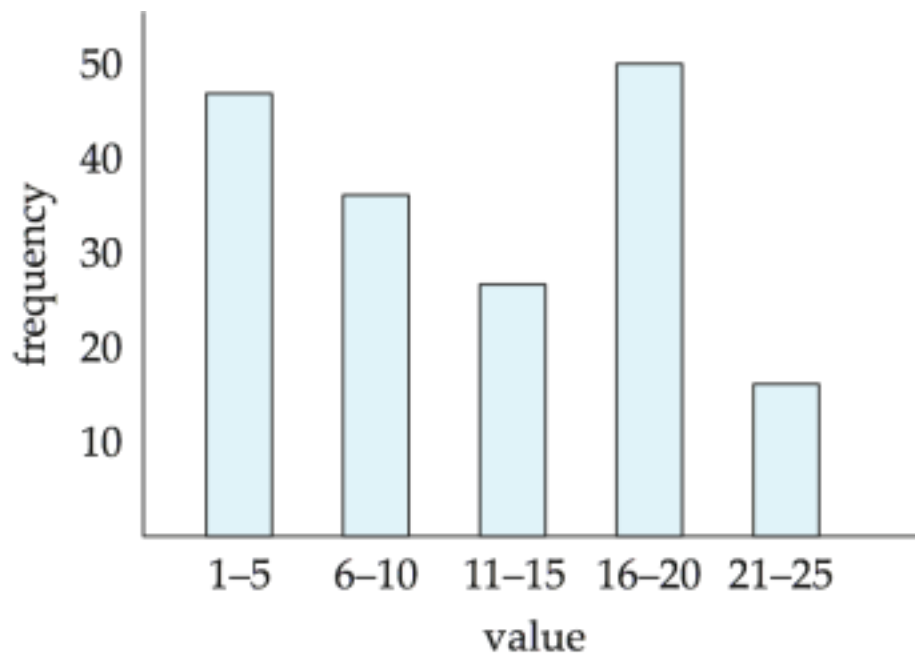
Estimation • n_r : number of tuples in a relation r .

- b_r : number of blocks containing tuples of r .
- l_r : size of a tuple of r .
- f_r : blocking factor of r — i.e., the number of tuples of r that fit into one block.
- $V(A, r)$: number of distinct values that appear in r for attribute A ; same as the size of $\Pi_A(r)$.
- If tuples of r are stored together physically in a file, then:

$$b_r = \lceil \frac{n_r}{f_r} \rceil$$

Histograms

- Histogram on attribute *age* of relation *person*



- **Equi-width** histograms
- **Equi-depth** histograms

Selection Size Estimation

- $\sigma_{A=v}(r)$
 - $n_r / V(A, r)$: number of records that will satisfy the selection
 - Equality condition on a key attribute: *size estimate* = 1
- $\sigma_{A \leq v}(r)$ (case of $\sigma_{A \geq v}(r)$ is symmetric)

- Let c denote the estimated number of tuples satisfying the condition.
- If $\min(A,r)$ and $\max(A,r)$ are available in catalog
 - $c = 0$ if $v < \min(A,r)$
 - $c = \frac{(\max(A,r) - \min(A,r)) \cdot n_r}{v - \min(A,r)}$
- If histograms available, can refine above estimate – In absence of statistical information c is assumed to be $n_r/2$.

Size Estimation of Complex Selections

- The **selectivity** of a condition θ_i is the probability that a tuple in the relation r satisfies θ_i .
 - If s_i is the number of satisfying tuples in r , the selectivity of θ_i is given by s_i/n_r .
- **Conjunction:** $\sigma_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n}(r)$. Assuming independence, estimate of c is:

tuples in the result

$$n_r * s_1 * s_2 * \dots * s_n$$

- **Disjunction:** $\sigma_{\theta_1 \vee \theta_2 \vee \dots \vee \theta_n}(r)$. Estimated number of tuples:

$$n_r * \left(1 - (1 - s_1) * (1 - s_2 * \dots * (1 - s_n) \right)$$

- **Negation:** $\sigma_{\neg \theta}(r)$. Estimated number of tuples:

$$n_r - \text{size}(\sigma_{\theta}(r))$$

Join Operation: Running Example

Running example:

student takes

Catalog information for join examples:

- $n_{\text{student}} = 5,000$.
- $f_{\text{student}} = 50$, which implies that

$$b_{student} = 5000/50 = 100.$$

- $n_{takes} = 10000.$
- $f_{takes} = 25$, which implies that
 $b_{takes} = 10000/25 = 400.$
- $V(ID, takes) = 2500$, which implies that on average, each student who has taken a course has taken 4 courses. – Attribute ID in $takes$ is a foreign key referencing $student$. – $V(ID, student) = 5000$ (primary key!)

Estimation of the Size of Joins

- The Cartesian product $r \times s$ contains $n_r \cdot n_s$ tuples; each tuple occupies $s_r + s_s$ bytes.
- If $R \cap S = \emptyset$, then $r s$ is the same as $r \times s$. • If $R \cap S$ is a key for R , then a tuple of s will join with at most one tuple from r – therefore, the number of tuples in $r s$ is no greater than the number of tuples in s .
- If $R \cap S$ in S is a foreign key in S referencing R , then the number of tuples in $r s$ is exactly the same as the number

of tuples in S .

- The case for $R \cap S$ being a foreign key referencing S is symmetric. •

In the example query *student takes*, *ID* in *takes* is a foreign key referencing *student*

– hence, the result has exactly n_{takes} tuples, which is 10000

Estimation of the Size of Joins (Cont.)

- If $R \cap S = \{A\}$ is not a key for R or S .

If we assume that every tuple t in R produces tuples in $R \cap S$, the number of tuples in $R \cap S$ is estimated to be: $n_r * n_s$

$$V(A, s)$$

If the reverse is true, the estimate obtained will

be: $n_r * n_s$

$$V(A, r)$$

The lower of these two estimates is probably the more accurate one.

- Can improve on above if histograms are available – Use

formula similar to above, for each cell of histograms on the two relations

Estimation of the Size of Joins (Cont.)

- Compute the size estimates for *depositor customer* without using information about foreign keys:
 - $V(ID, takes) = 2500$, and
 $V(ID, student) = 5000$
 - The two estimates are $5000 * 10000/2500 = 20,000$ and $5000 * 10000/5000 = 10000$
 - We choose the lower estimate, which in this case, is the same as our earlier computation using foreign keys.

Size Estimation for Other Operations

- Projection: estimated size of $\Pi_A(r) = V(A, r)$

Aggregation : estimated size of $_A$

$$g_F(r) = V(A, r)$$

- Set operations

- For unions/intersections of selections on the same relation:

rewrite and use size estimate for selections

- E.g. $\sigma_{\theta_1}(r) \cup \sigma_{\theta_2}(r)$ can be rewritten as $\sigma_{\theta_1} \sigma_{\theta_2}(r)$
- For operations on different relations:
 - estimated size of $r \cup s = \text{size of } r + \text{size of } s$.
 - estimated size of $r \cap s = \text{minimum size of } r \text{ and size of } s$.
 - estimated size of $r - s = r$.
 - All the three estimates may be quite inaccurate, but provide upper bounds on the sizes.

Size Estimation (Cont.)

- Outer join:
 - Estimated size of $r \bowtie s = \text{size of } r \cdot s + \text{size of } r$ • Case of right outer join is symmetric
 - Estimated size of $r \bowtie s = \text{size of } r \cdot s + \text{size of } r + \text{size of } s$

Estimation of Number of Distinct

Values Selections: $\sigma_{\theta}(r)$

- If θ forces A to take a specified value: $V(A, \sigma_{\theta}(r)) = 1$.
 - e.g., $A = 3$
- If θ forces A to take on one of a specified set of values:

$V(A, \sigma_{\theta}(r))$ = number of specified values. • (e.g., $(A = 1 \vee A = 3 \vee A = 4)$),

- If the selection condition θ is of the form $A \text{ op } r$
estimated $V(A, \sigma_{\theta}(r)) = V(A.r) * s$
 - where s is the selectivity of the selection.
- In all the other cases: use approximate estimate of $\min(V(A, r), n_{\sigma_{\theta}(r)})$
 - More accurate estimate can be got using probability theory, but this one works fine generally

Estimation of Distinct Values (Cont.)

Joins: $r \bowtie s$

- If all attributes in A are from r
estimated $V(A, r \bowtie s) = \min(V(A, r), n_{rs})$
- If A contains attributes $A1$ from r and $A2$ from s , then
estimated
 $V(A, r \bowtie s) =$
 $\min(V(A1, r) * V(A2 - A1, s), V(A1 - A2, r) * V(A2, s), n_{rs}) -$

More accurate estimate can be got using probability theory, but this one works fine generally

Estimation of Distinct Values (Cont.)

- Estimation of distinct values are straightforward for projections.
 - They are the same in $\Pi_{A(r)}$ as in r .
- The same holds for grouping attributes of aggregation.
- For aggregated values
 - For $\min(A)$ and $\max(A)$, the number of distinct values can be estimated as $\min(V(A,r), V(G,r))$ where G denotes grouping attributes
 - For other aggregates, assume all values are distinct, and use $V(G,r)$

Additional Optimization Techniques

- Nested Subqueries

■ Materialized Views

Optimizing Nested Subqueries**

- Nested query example:

```
select name  
from instructor  
where exists (select *  
  from teaches  
  where instructor.ID = teaches.ID and  
    teaches.year = 2007)
```

- SQL conceptually treats nested subqueries in the `where` clause as functions that take parameters and return a single value or set of values
 - Parameters are variables from outer level query that are used in the nested subquery; such variables are called **correlation variables**
- Conceptually, nested subquery is executed once

for each tuple in the cross-product generated by the outer level **from** clause

- Such evaluation is called **correlated evaluation**
- Note: other conditions in where clause may be used to compute a join (instead of a cross-product) before executing the nested subquery

Optimizing Nested Subqueries (Cont.)

- Correlated evaluation may be quite inefficient since
 - a large number of calls may be made to the nested query – there may be unnecessary random I/O as a result
- SQL optimizers attempt to transform nested subqueries to joins where possible, enabling use of efficient join techniques
- E.g.: earlier nested query can be rewritten as
select *name*
from *instructor, teaches*
where *instructor.ID = teaches.ID and teaches.year = 2007*

– Note: the two queries generate different numbers of duplicates (why?)

- teaches can have duplicate IDs
- Can be modified to handle duplicates correctly as we will see

- In general, it is not possible/straightforward to move the entire nested subquery from clause into the outer level query from clause
 - A temporary relation is created instead, and used in body of outer level query

Optimizing Nested Subqueries (Cont.)

In general, SQL queries of the form below can be rewritten as shown

- Rewrite: **select ...**
from L_1
where P_1 and exists (select *
from L_2
where P_2)

- To: **create table t_1 as**
select distinct V
from L_2
where P_{2^1}

select ...
from L_1, t_1

- P_{2^1} contains predicates in P_2 that do not involve

where P_1 and P_{2^2}

any correlation variables

- P_{2^2} reintroduces predicates involving correlation variables, with relations renamed appropriately
- V contains all attributes used in predicates with correlation variables

Optimizing Nested Subqueries (Cont.)

- In our example, the original nested query would be transformed to
create table t_1 as
select distinct ID
from $teaches$
where $year = 2007$

select *name*
from *instructor*, t_1
where $t_1.ID = instructor.ID$

- The process of replacing a nested query by a query with a join (possibly with a temporary relation) is called **decorrelation**.

-

Optimizing nested queries...cont

- Decorrelation is more complicated when
 - the nested subquery uses aggregation, or
 - when the result of the nested subquery is used to test for equality, or
 - when the condition linking the nested subquery to the other query is **not exists**,
 - and so on.

- A **materialized view** is a view whose contents are computed and stored.
- Consider the view
create view *department_total_salary*(*dept_name*,
total_salary) **as**
select *dept_name*, **sum**(*salary*)
from *instructor*
group by *dept_name*
- Materializing the above view would be very useful if the total salary by department is required frequently
 - Saves the effort of finding multiple tuples and adding up their amounts

Materialized View Maintenance

- The task of keeping a materialized view up-to-date with the underlying data is known as **materialized view maintenance**

- Materialized views can be maintained by recomputation on every update
- A better option is to use **incremental view maintenance** –
Changes to database relations are used to compute changes to the materialized view, which is then updated
- View maintenance can be done by
 - Manually defining triggers on insert, delete, and update of each relation in the view definition
 - Manually written code to update the view whenever database relations are updated
 - Periodic recomputation (e.g. nightly)
 - Above methods are directly supported by many database systems
 - Avoids manual effort/correctness issues

Incremental View Maintenance

- The changes (inserts and deletes) to a relation or expressions are referred to as its **differential**
 - Set of tuples inserted to and deleted from r are denoted i_r and d_r

- To simplify our description, we only consider inserts and deletes
 - We replace updates to a tuple by deletion of the tuple followed by insertion of the update tuple
- We describe how to compute the change to the result of each relational operation, given changes to its inputs
- We then outline how to handle relational algebra expressions

Join Operation

- Consider the materialized view $v = r \Join s$ and an update to r
- Let r^{old} and r^{new} denote the old and new states of relation r
- Consider the case of an insert to r :
 - We can write r^{new} as $(r^{old} \cup i_r)$
 - And rewrite the above to $(r^{old} \Join s) \cup (i_r \Join s)$
 - But $(r^{old} \Join s)$ is simply the old value of the materialized view, so the incremental change to the view is just $i_r \Join s$
- Thus, for inserts $v^{new} = v^{old} \cup (i_r \Join s)$
- Similarly for deletes $v^{new} = v^{old} - (d_r \Join s)$

		C, 2	2, s
		1, p 2, r 2, s	C, 2, r C, 2, s
A, 1 B, 2		A, 1, p B, 2, r B,	

Selection and Projection Operations

- Selection: Consider a view $v = \sigma_{\theta}(r)$.
 - $v^{new} = v^{old} \cup \sigma_{\theta}(i_r)$
 - $v^{new} = v^{old} - \sigma_{\theta}(d_r)$
- Projection is a more difficult operation
 - $R = (A, B)$, and $r(R) = \{ (a, 2), (a, 3) \}$
 - $\Pi_A(r)$ has a single tuple (a) .
 - If we delete the tuple $(a, 2)$ from r , we should not delete the tuple (a) from $\Pi_A(r)$, but if we then delete $(a, 3)$ as well, we should delete the tuple
- For each tuple in a projection $\Pi_A(r)$, we will keep a count of how many times it was derived
 - On insert of a tuple to r , if the resultant tuple is already in $\Pi_A(r)$ we increment its count, else we add a new tuple with count = 1

- On delete of a tuple from r , we decrement the count of the corresponding tuple in $\Pi_A(r)$

Aggregation Operations

- **count** : $v = \mathcal{A}g_{count(B)}^{(r)}$.
 - When a set of tuples i_r is inserted
 - For each tuple r in i_r , if the corresponding group is already present in v , we increment its count, else we add a new tuple with count = 1
 - When a set of tuples d_r is deleted
 - for each tuple t in i_r , we look for the group $t.A$ in v , and subtract 1 from the count for the group.
 - If the count becomes 0, we delete from v the tuple for the group $t.A$
- **sum**: $v = \mathcal{A}g_{sum(B)}^{(r)}$
 - We maintain the sum in a manner similar to count, except we add/subtract the B value instead of adding/subtracting 1 for the count
 - Additionally we maintain the count in order to detect groups with no tuples. Such groups are deleted from v
 - Cannot simply test for sum = 0 (why?)
- To handle the case of **avg**, we maintain the **sum** and **count** aggregate values separately, and divide at the end

Aggregate Operations (Cont.)

- **min, max:** $v = \underset{A}{g_{\min(B)}(r)}$.
 - Handling insertions on r is straightforward.
 - Maintaining the aggregate values **min** and **max** on deletions may be more expensive. We have to look at the other tuples of r that are in the same group to find the new minimum

Other Operations

- **Set intersection:** $v = r \cap s$
 - when a tuple is inserted in r we check if it is present in s , and if so we add it to v .
 - If the tuple is deleted from r , we delete it from the intersection if it is present.
 - Updates to s are symmetric
 - The other set operations, *union* and *set difference* are handled in a similar fashion.
- Outer joins are handled in much the same way as joins but with some extra work

- we leave details to you.

Handling Expressions

- To handle an entire expression, we derive expressions for computing the incremental change to the result of each sub-expressions, starting from the smallest sub-expressions. • E.g. consider $E_1 E_2$ where each of E_1 and E_2 may be a complex expression
 - Suppose the set of tuples to be inserted into E_1 is given by D_1
 - Computed earlier, since smaller sub-expressions are handled first – Then the set of tuples to be inserted into $E_1 E_2$ is given by $D_1 E_2$
 - This is just the usual way of maintaining joins

Query Optimization and Materialized

Views • Rewriting queries to use materialized views:

- A materialized view $v = r s$ is available
- A user submits a query $r s t$
- We can rewrite the query as $v t$
 - Whether to do so depends on cost estimates for the two alternative

- Replacing a use of a materialized view by the view definition:
 - A materialized view $v = r \Join s$ is available, but without any index on it
 - User submits a query $\sigma_{A=10}(v)$.
 - Suppose also that s has an index on the common attribute B , and r has an index on attribute A .
 - The best plan for this query may be to replace v by $r \Join s$, which can lead to the query plan $\sigma_{A=10}(r) \Join s$
- Query optimizer should be extended to consider all above alternatives and choose the best overall plan

Materialized View Selection

- **Materialized view selection:** “What is the best set of views to materialize?”.
- **Index selection:** “what is the best set of indices to create” – closely related, to materialized view selection
 - but simpler

- Materialized view selection and index selection based on typical system **workload** (queries and updates) – Typical goal: minimize time to execute workload , subject to constraints on space and time taken for some critical queries/updates
 - One of the steps in database tuning
 - more on tuning in later chapters
- Commercial database systems provide tools (called “tuning assistants” or “wizards”) to help the database administrator choose what indices and materialized views to create